

# CSS level-1

## \* ~~Text-decoration~~ <sup>align</sup> -

①

• para {

~~text-decoration~~ <sup>align</sup> : center;  
left;  
right;

}

Hello

Hello

Hello

②

\* para {

~~text-decoration~~ <sup>align</sup> : start;  
end;

}

left justify

right justify

## \* Text-decoration :

\* para {

text-decoration : underline;  
overline;  
line-through;  
none;  
red underline;

}

Hello

Hello

Hello

Hello

refer To mpn for more.

\* font-weight :

normal

bold

bolder

lighter

font-weight = 100 - 900

lightest

100

boldest

900

<h1> Hello </h1>

h1 {

font-weight : light ;

}

Hello

→

Hello

Yes to hunxa

\* font-family :

{

font-family :

}

→ google fonts herney.

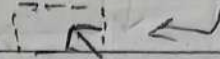
;

~~✗~~

## Units in CSS

1. Absolute units :- ( 1 inch = 96px ) = 2.54cm

Browser → inspect click on the most left icon in inspect which is like this



{

font-size : 20px ;

}

2. Text properties :-

i) Line-height :-

{

line-height : 2px ;

}

Hello } → 2px  
 Hello } → 5px  
 Sahil }

ii) text-transform :-

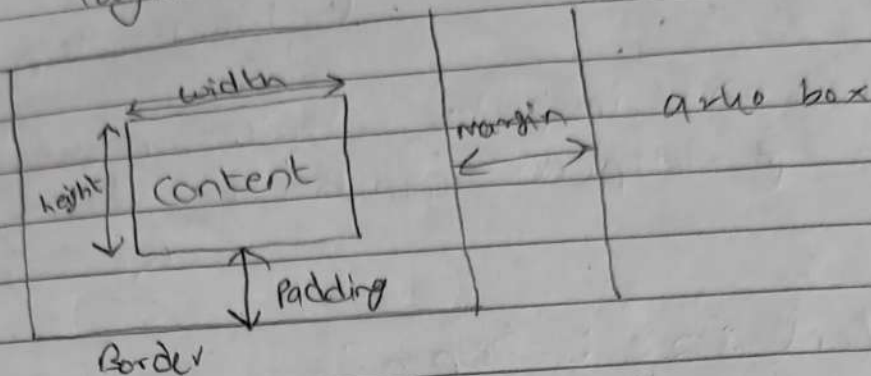
{

text-transform : uppercase / lowercase / capitalize / none ;

}

## level - 2 CSS

### \* Box layout

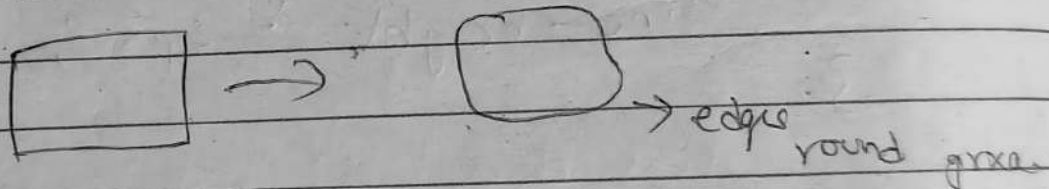


### \* Border :-

width &

style - solid, dotted, double.

### \* Border radius :-



### To create circle :-

border radius : 50%, ;

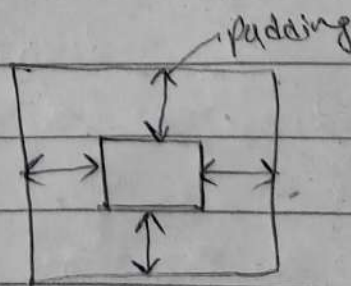
### \* Padding :

padding-left ;

- right ;

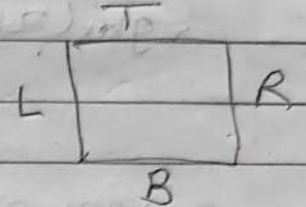
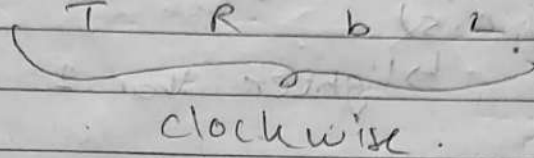
- top ;

- bottom ;



padding : 50px ; [ Top, right, bottom, left ]  
sab ra 50px ;

padding : 1px 2px 3px 4px



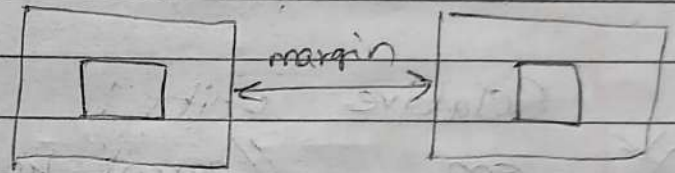
## \* margin

margin-left

margin-right

margin-top

margin-bottom



margin : 1px 2px 3px 4px

## \* display :

inline : chahinay jati ntra space linxa.

block : pura line, sab width 100% space linxa.

inline-block : inline jeto ho but width or height milauna milxa.

none : content nae hatae dinxa.

## \* visibility :

hidden : same as display none tara element  
hatey ni space allocated hatdaena.

\* rgba (red, green, blue, )

rgba(255, 0, 0, 1)

↳ strongest red

rgba(255, 0, 0, 0.5)

↳ lighter than 1

- ranges from (0 to 1)
- 0 is lightest
- 1 is darkest

Relative units:

\* em //parent ko anushar

box1 width  $\rightarrow$  25px;

box2 width  $\rightarrow$  2em;  $\rightarrow$  2xparent i.e. box-1  
 $= 50px$

Scenario: -

<div id="box1"> box1 &

<div id="box2"> <sup>box2</sup> </div>

</div>

\* Root em (rem):

- root element ko anushar
- sabse first element ko size anushar change hunkar

Hello  $\rightarrow$  16px

$\rightarrow$  5rem  $= 5 \times 16 = 80px$

\* vh → viewport height:-  
↓  
browser

50vh  $\rightarrow$  50% of browser height.

$vw \rightarrow$  viewport width :-  
↓  
browser

Position :-

static - default position (The top, right, bottom, left, z-index properties have no effect)

relative - element is relative to itself  
(top, r, b, l, z-index will work)

absolute - positioned relative to its closest ancestor

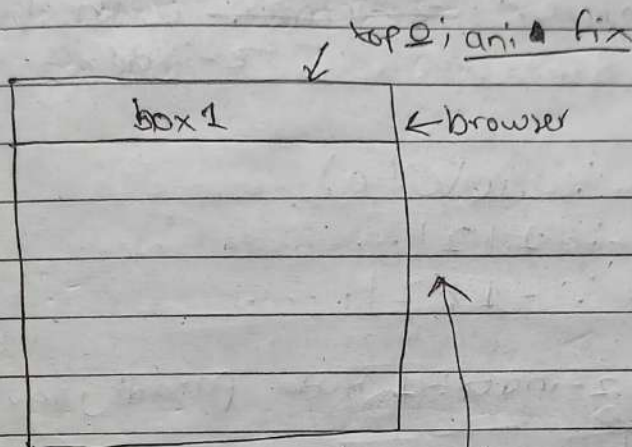
fixed - relative to browser. (removed from flow)

sticky - positioned based on user's scroll position.



- nearest parent body bhayo tei anushar change hunxa.
- top, ri, left, bot sab value tei anushar lekha milxa.

④ fixed:



\* box 1 {

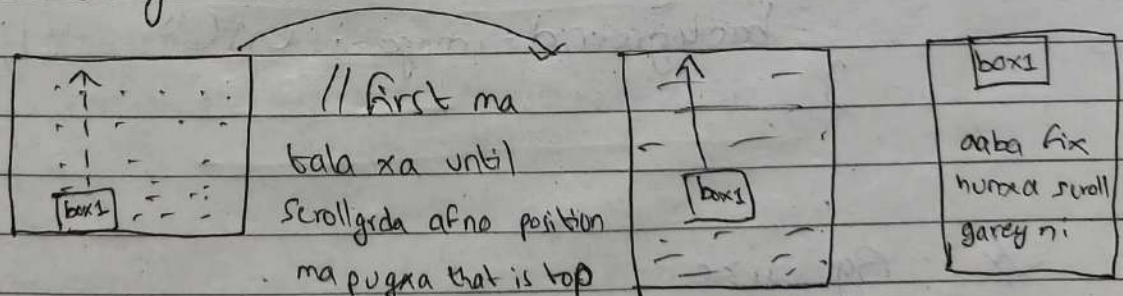
position: fixed;

top: 0;

width: 100vw;

}

⑤ Absolute sticky:-



\* box 1 {

position: sticky;

top: 0;

}

//static ma chaldaera

z-index : ←

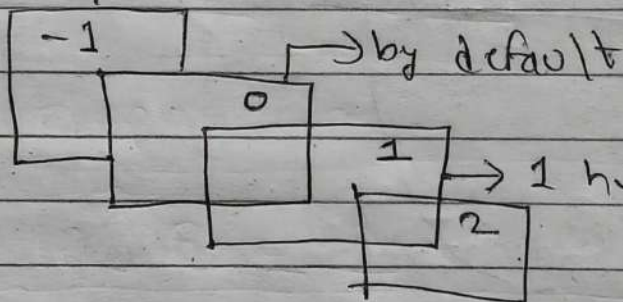
- It decides the stack level of elements.
- elements with larger z-index will cover <sup>elements</sup> ~~z-index~~ with low z-index.

z-index : auto(0)

z-index : 1 | 2 | ...

z-index : -1 | -2 | ...

→ z-index: -1 huda paradi janxa



✗ Background image:-

```
{  
  background-image : url(" ");  
}
```

✗ Bg size

① cover

② contain

③ auto.

① cover : // most used.

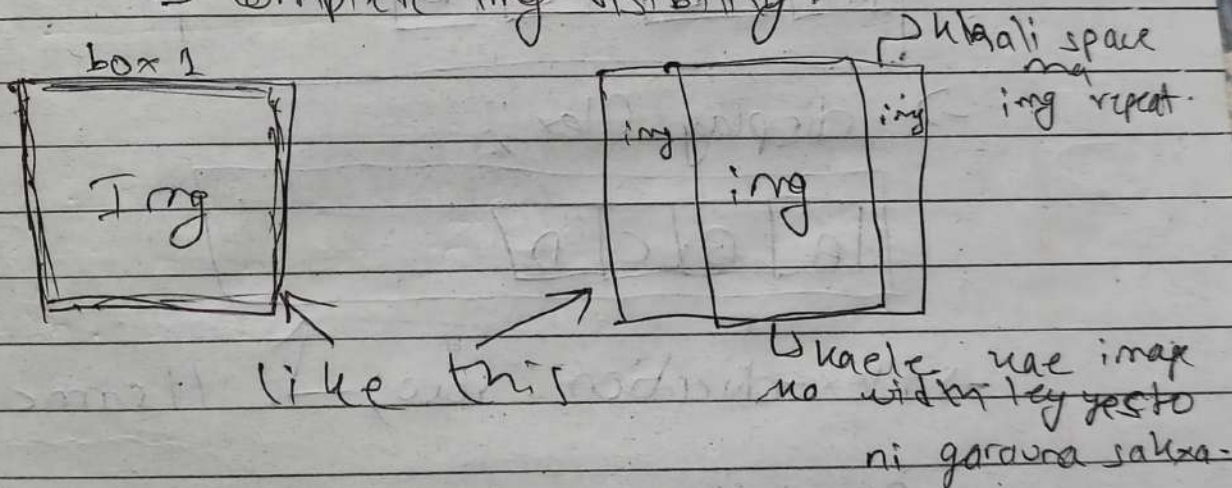
```
{
background-size : cover;
}
```

- ~~image~~ <sup>div</sup> na ke pani empty space bardhena
- sab cover hunxa ing. ky.

② contain : // not used mostly cases na.

→ fits the div

→ image visibility na focus garxa  
↳ complete ing visibility.



```
{
```

```
background-size : contain;
background-repeat : no-repeat;
```

③ auto :

by default auto me hunxa.

① cover : // most used:

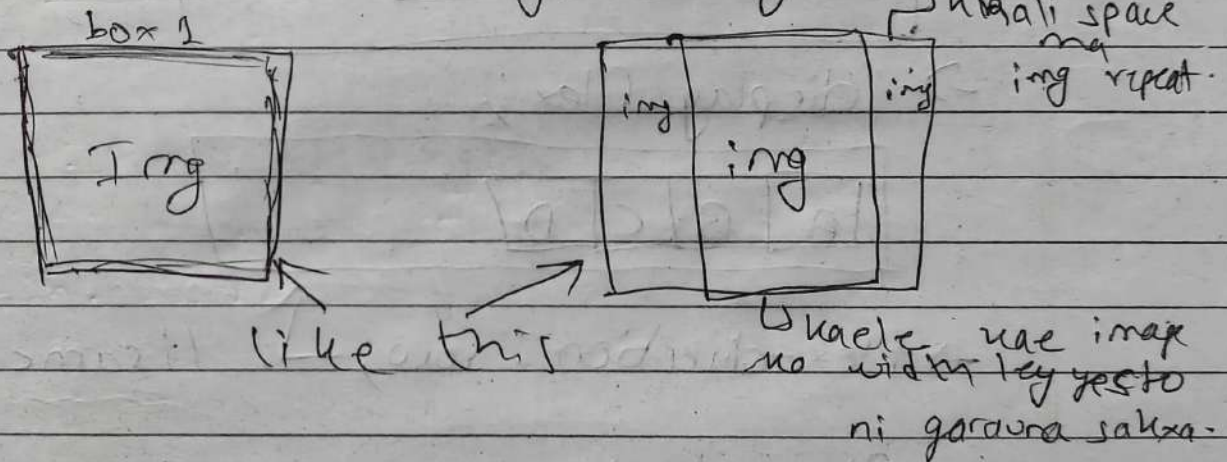
```
{
background-size : cover;
}
```

- ~~image~~ <sup>div</sup> ma ke pani empty space bahana
- sab cover hunxa ing. ky.

② contain : // not used mostly cases ma.

→ fits the div

→ image visibility ma focus garxa  
↳ complete img visibility.



```
{
```

```
background-size : contain;
background-repeat : no-repeat;
```

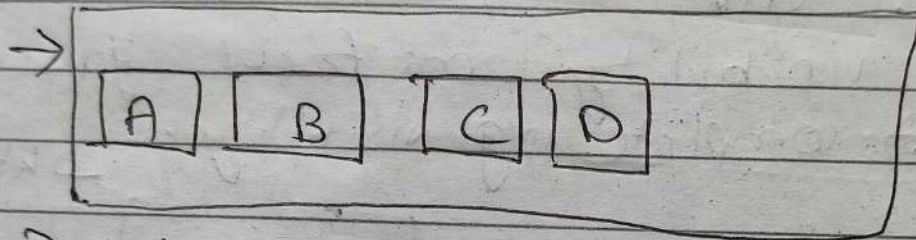
③ auto :

by default auto me hunxa.

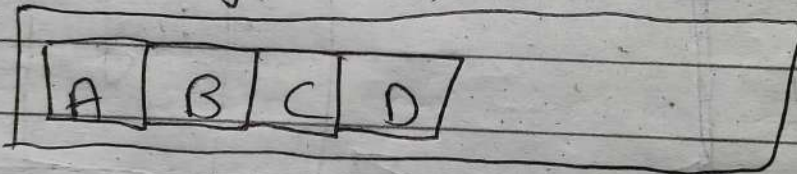
# Flex Box

## ① Flexbox Direction:

- flexbox-direction: row; (default)
- flexbox-direction: column;
- flexbox-direction: row-reverse;
- flexbox-direction: column-reverse;

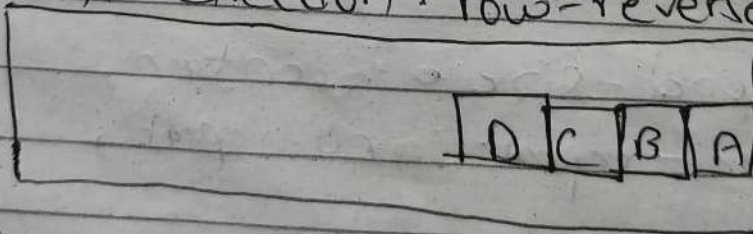


→ display flex;

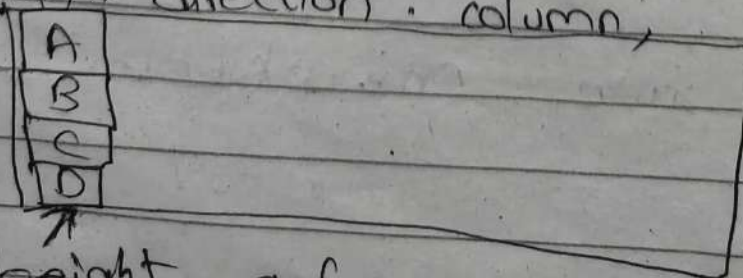


→ flex-direction: row; ↑ // same

→ flex-direction: row-reverse;



→ flex-direction: column;



height  
shrink  
qafae  
blawo.

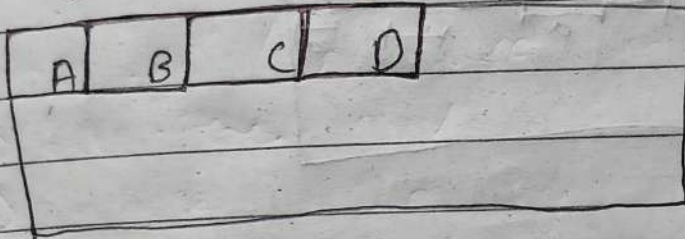
## Most used flex properties:-

(2) • justify-content: alignment along main axis.

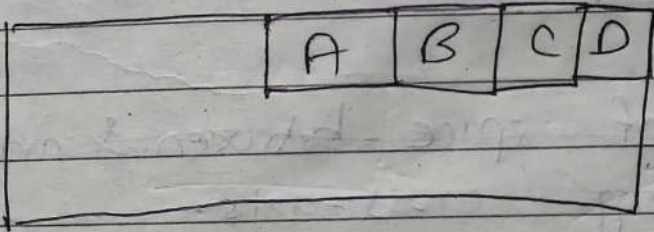
flex-start / flex-end / centre / space-around /

1. flex-start :

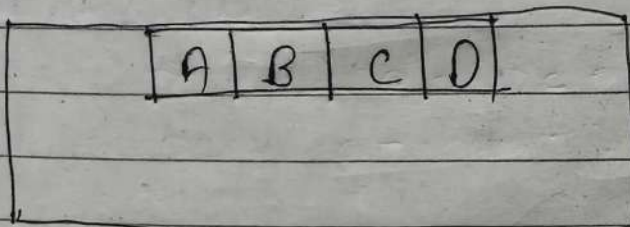
justify-content : flex-start ;



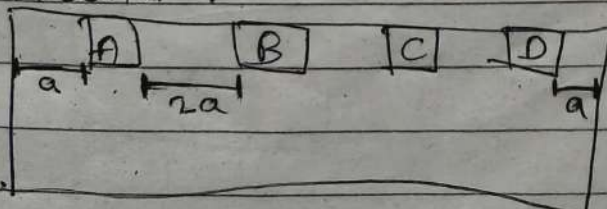
2. flex-end :



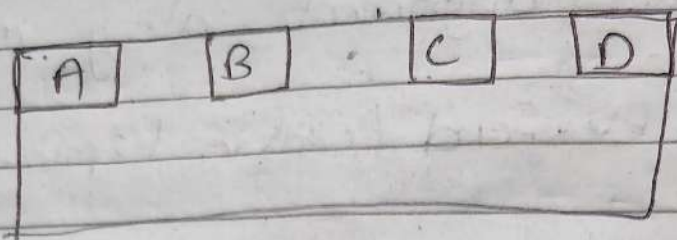
3. centre :



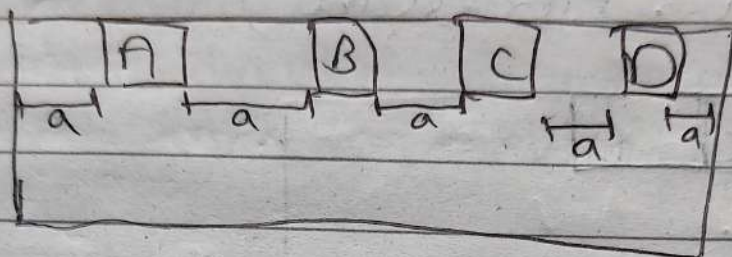
4. space-around :



5. Space-between:



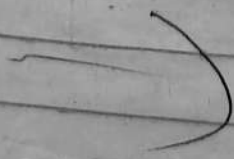
6. space-evenly:



3

align-content:

- alignment of space between & around the content along cross-axis.



### ③ Flex-wrap :-

Sets whether flex items are forced onto one line or can wrap onto multiple lines.

- flex-wrap : nowrap; // by default.
- flex-wrap : wrap;
- flex-wrap : wrap-reverse;

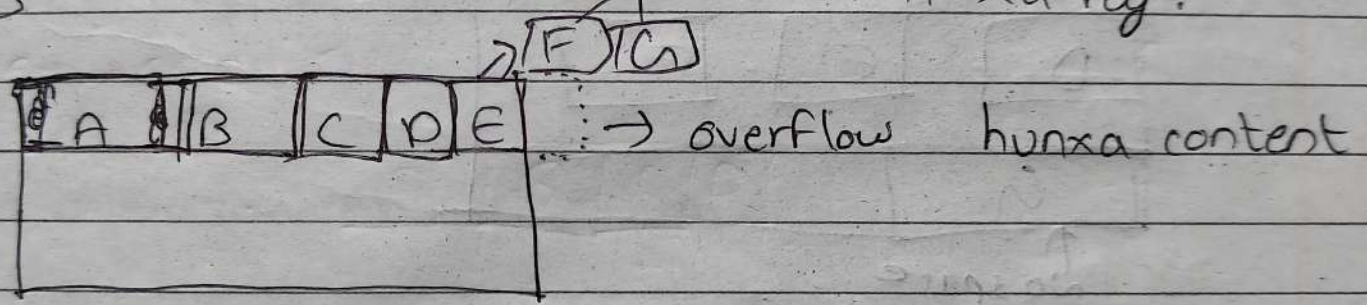
div {

width: 300px;

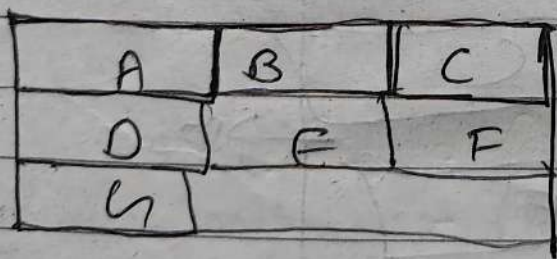
height: 100px;

}

yo div ta ni xa rey.

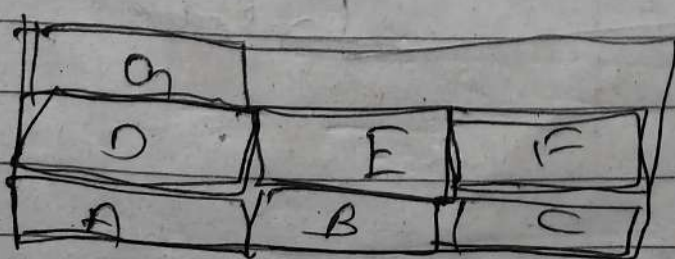


- flex-wrap : wrap;



Yesani milxa.

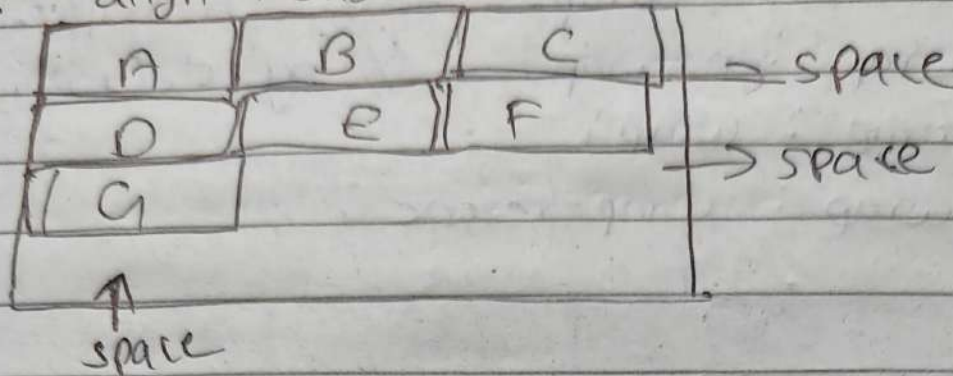
- flex-wrap : wrap-reverse;



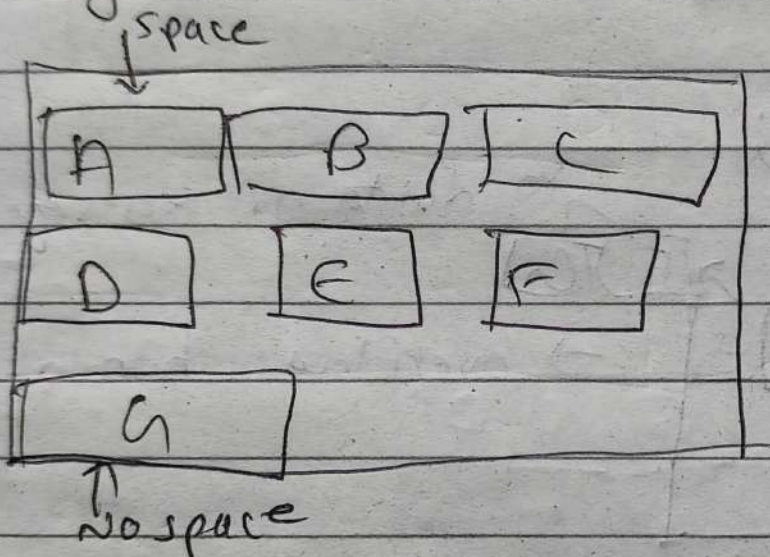
## ④ Align Items:

Distributes our item along the cross axis.

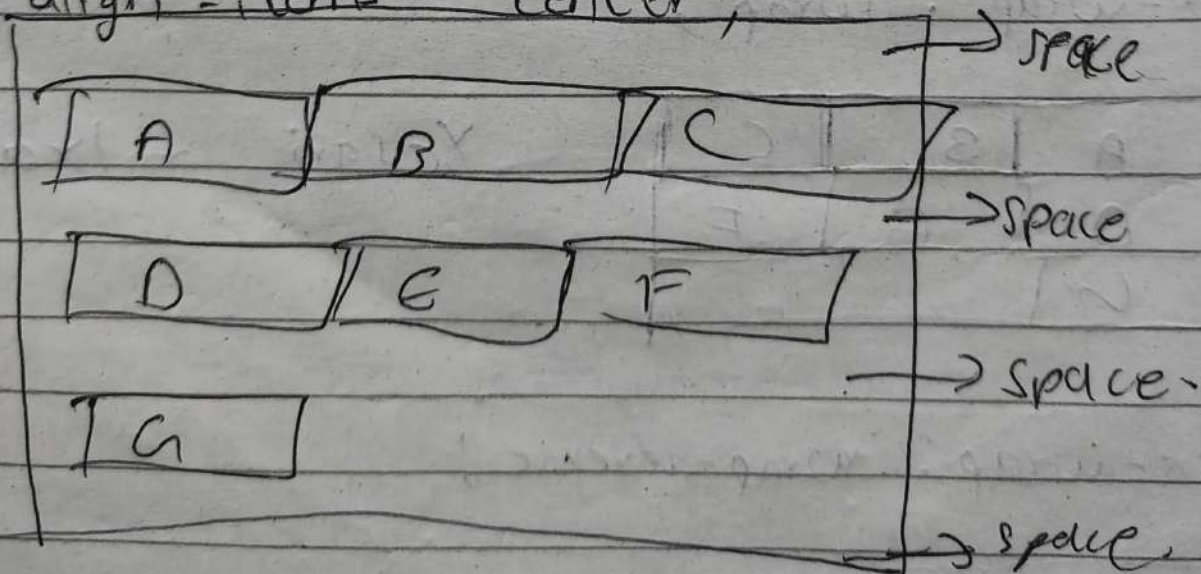
1. align-items: flex-start;



2. align-items: flex-end;

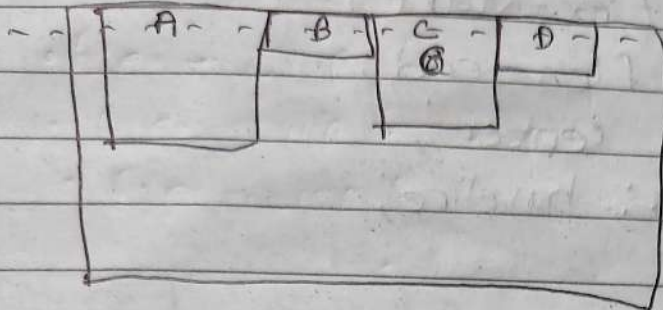


3. align-items: center;



#### 4. Baseline

align-items: baseline;

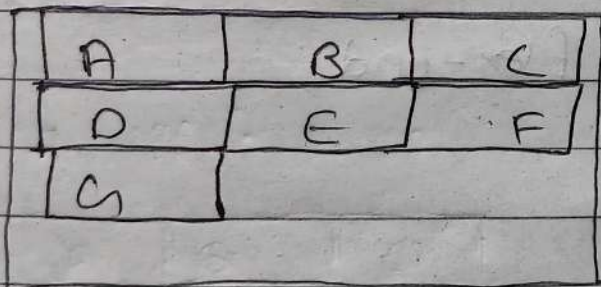


content no <sup>anushar</sup> same line <sup>base</sup> despite the size.

#### 5) Align-Content :-

- sets the distribution of space between and around content items along a flexbox's cross-axis.

1. align-content: flex-start;



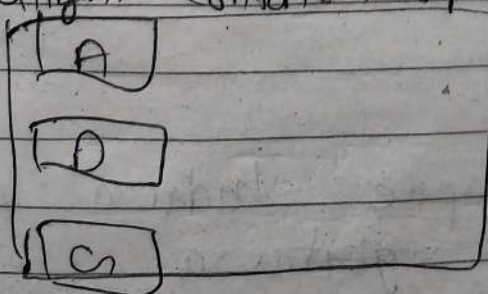
// no space between lines

// all same as <sup>Align</sup> flex-items.

2. align-content: flex-end;

3. align-content: center;

4. align-content: space-between; // space-around; space-evenly; baseline;

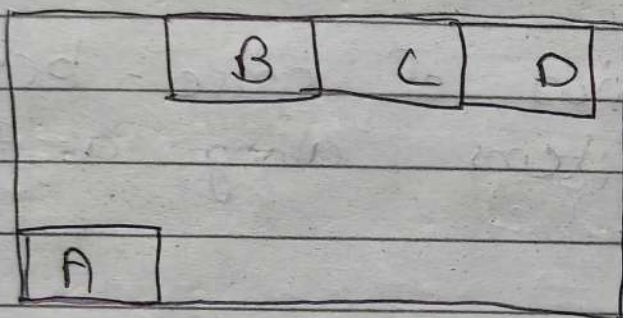


## ⑥ Align-self :-

Aligns an item along the cross axis.

- align-self : flex-start;
- " " : flex-end;
- " " : center;
- " " : baseline;

• align-self : flex-end;



~~✗~~ A {

align-self : flex-end;  
}

## ⑦ Flex Sizing :-

- flex-basis

It sets the initial main size of a flex item.

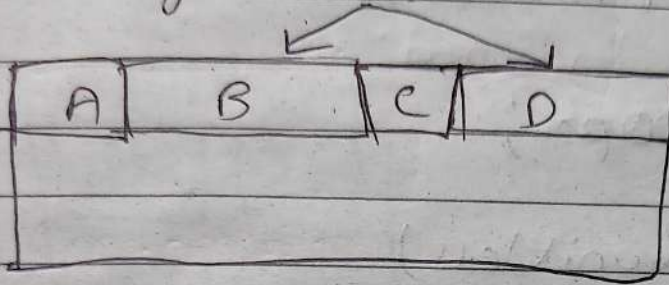
flex-basis : 100px;

// thau xa bhney space kidauxa nbra  
ghatauxa.

## • flex-grow

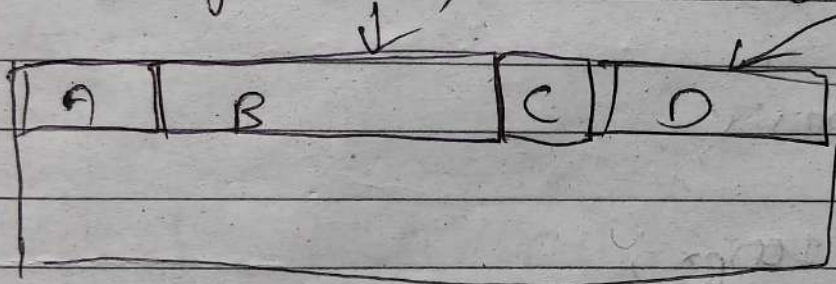
It specifies how much of the flex container's remaining space should be assigned to the flex item's main size.

flex-grow : 1



flex-grow : 2 ;

flex-grow : 1 ;



max-width : 400px ;



yesky limitation laedina flex-grow : 2 ; nae bhaye

-i 400px ; max-width banaedina.

## • flex-shrink : 1 ;

2  
3

Overflow huna didana ;

control gna min-width : ... ;

## ⑧ Flex Short hand :-

- flex-grow / flex-shrink / flex-basis

→ flex : 2 2 100px ;

- flex-grow / flex-basis

→ flex : 2 100px ;

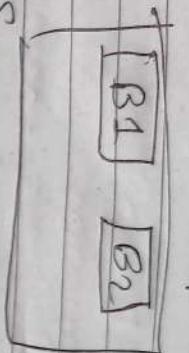
- flex-grow (unitless)

→ flex : 2 ;

- flex-basis

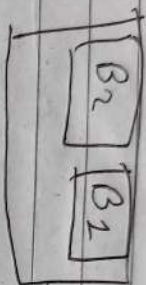
→ flex : 100px ;

# Order property



\*B1 {

order: 2;



## Media queries

@media (width: 600px) { // exactly 600px

div {

background-color: red; // exactly 600px  
background-color: red; // exactly 600px

}

@media (min-width: 600px) { // 600px + honey

div {

background-color: green; // 600px + honey  
background-color: green; // 600px + honey

}

}

(max-width: 600px) { // 600px - honey

background-color: green; // 600px - honey

Date \_\_\_\_\_  
Page \_\_\_\_\_

@media (min-width : 200px) and (max-width : 300px)

→ Hover :

```
div : hover {  
    background-color : red;  
    color : white;  
}
```

```
div : active { // click & hold mouse cursor.  
    background-color : blue;  
}
```

## \* Transitions

Transitions enable you to define the transition between two states of an element.

- transition - property : property you want to transition (font-size, width etc)
- transition - duration : 2s / 4ms
- transition - timing-function : ease-in / ease-out / linear / steps
- transition - delay : 2s / 4ms ..